

C323 / Spring 2026 Final Project

Team Members: Jack Snyder (snyderjk@iu.edu), Trevor Lough (tlough@iu.edu) [Team 8]

App Name: Fitness Tracker App \ **IU GitHub Submission Date:** May 5, 2026

Quick Notes

- CoreMotion step counting and activity detection require a physical iPhone; the simulator will show fallback text ("Not available on simulator") for activity type, which is expected behavior per Apple's documentation. If app does not run when first downloaded, "Signing and Capabilities in the XCode app targets need CoreMotion to give a message to send to the user's iPhone to notify that motion is being tracked. If this is empty, feel free to add whatever so it runs. Our default is "Running workout is being tracked."
 - App tested on iPhone 16 Simulator and physical iPhone 15 Pro — all views render correctly and evenly.
-

Features Present in Our App

Full UI Functionality

- **WorkoutView:** Main view on app start. Able to start a new workout via the "+ Add Exercise" button. Each workout can contain multiple exercises each with one or multiple sets within them. "Save Workout" button in green when finished.
- **HistoryView:** Shows your past workouts that you have logged, along with a date and a more detailed view when clicked on. You can edit your workouts by tapping "Edit" up top and deleting a workout, or you can slide left to bring up a delete functionality. This is our "Table View" for the app.
- **ProgressView:** Shows your progress through each unique exercise you have logged. Shows your best "max" weight you have done, where you started at, and how many sessions you have done with that selected exercise from the drop-down. All accompanied with a progress line graph made using CoreGraphics.
- **MeasurementView:** Independent view to document your body measurements and the progress you have made with your weight. You can log your weight (in lbs) and your body fat as a percentage (which is optional). The drop-down below this also includes more optional fields (chest, waist, arms). Below the log card is a graph with your weight trend and the history of your logs with the date and information submitted.
- **AnalyticsView:** Many different statistics that follow your workouts you have logged in your Workout View. Top left card shows the total number of workouts you have logged, top right shows the streak of how many days you have logged workouts in a row, below is also total volume and total sets completed. Below these 4 is a line graph with your workout frequency measured in how many workouts per week. Scrolling below this reveals your most frequently used exercise and your personal records for each unique exercise logged. All views embedded in a tab view named: **MainTabView**.

MVC Architechural Pattern

Our app follows the M-V-C pattern equivalent (MVVM) in SwiftUI. Our controllers are ViewModels and have the same sorts of functionalities that controllers have in other programs that follow this pattern.

Persistent Storage

All workout and measurement data is now saved between app launches using `UserDefaults` with JSON encoding/decoding via `Codable`. Both `WorkoutViewModel` and `MeasurementViewModel` save on every write and load on init.

CoreMotion Integration (Feature shown in class)

Added `MotionManager.swift` using `CMPPedometer` and `CMMotionActivityManager` from the `CoreMotion` framework. When a workout session is active (first exercise added), the app begins tracking:

- Live step count
- Current activity type (running, walking, cycling, stationary) These are displayed in a live banner in `WorkoutView`.

CoreHaptics (Not shown in class)

`HapticManager` is triggered at every appropriate user interaction — saving workouts, adding exercises, adding sets, saving measurements, and deleting entries. This is found as its own separate file and has instances in multiple views.

CoreGraphics/Swift Charts (Shown in class)

`MeasurementView` & `AnalyticsView` use CoreGraphics as the main graph functionality. `ProgressView`, however, uses the Swift Charts library. Both are present as Charts was the original implementation but due to the project requirements, and that Charts uses CoreGraphics underneath the hood, we opted to use CG as well.

Combine (Not shown in class)

Used to handle asynchronous events, data streams, and UI updates. Present in all ViewModels.

Swift Conventions

All files include team member headers with names, emails, app name, and submission date. All types, properties, and methods follow Swift naming and capitalization conventions. Documentation comments added to all public types and key methods.

Changes from Original Design

- Models made Codable for persistence (no design change, implementation upgrade)
- DashboardView and DashboardCard removed (replaced by direct tab navigation per revised design)
- MotionManager.swift added as new file
- HistoryView workouts now tap through to a WorkoutDetailView showing all exercises and sets. Empty state shown when no workouts exist. Sorted most-recent-first.

- Most other functionalities the same with added features discussed in wording from original design but not present on UML diagrams.
-

Individual Contributions + Responsibilities

- **Jack Snyder:** WorkoutView + CoreMotion integration, HistoryView detail navigation, Models (Codable), WorkoutViewModel persistence, MainTabView, HapticManager
- **Trevor Lough:** MeasurementView expanded fields + graph, ProgressView picker + Swift Charts, AnalyticsView grid layout + stats, MeasurementViewModel persistence, AnalyticsViewModel + ProgressViewModel